PharmaWatch

Technical interview preparation guide

Architecture · Big data · Statistics · Machine learning · Question bank

How to use this guide

- Sections 1–9 explain *what was built and why*. Section 10 is a bank of interview questions with model answers. Section 11 is a one-page cheat sheet.
- Green boxes are things to say. Amber boxes are weaknesses you should volunteer. Red boxes are interviewer traps.
- Every claim here was checked against the code and the project's own planning documents. Where a feature is planned and not built, the guide says so.
- If you only have an hour: read sections 1, 2, 5, 7.1 and 9, then skim 10 and 11.

#	Contents
1	The project in one page (pitch, audience, key facts)
2	Architecture (layers, request flow, why a monolith)
3	Tech stack and the reasoning for each choice
4	Big data: FAERS pipeline, Kafka, HBase, what is simulated
5	Statistics: PRR, ROR, chi-square, IC, signal rules, limits
6	Identity: RxNorm and MedDRA
7	Machine learning: GINE DDI, baselines, LSTM, NER, OCR
8	Engineering quality, security, current status
9	Your ten weak spots and how to answer them
10	Interview question bank (Q&A)
11	Cheat sheet, glossary, last-night checklist

Based on the repository at branch feat/milestone-2-pilot. Research and education tool, not a clinical system.

1. The project in one page

SAY THIS

30-second version. “PharmaWatch is a drug-safety monitoring dashboard. It takes the FDA's public adverse-event reports (FAERS), cleans and de-duplicates them in a columnar database, and uses standard pharmacovigilance statistics (PRR, ROR, chi-square, IC) to flag drug–event pairs that are reported more often than expected. On top of that it has a graph neural network that predicts drug–drug interactions from molecular structure, an LSTM for report-trend forecasting, and a biomedical NER model that mines free text. Data arrives through Kafka, interaction knowledge sits in HBase, and the UI is a CDC-styled web app. It is a research and education tool, not a clinical decision system.”

2-minute version (add these points in order): problem → data → statistics → ML → big-data plumbing → what you learned and what is still imperfect. Always finish with the honest limitations; interviewers reward that far more than polish.

The problem it addresses

- Drugs are tested on thousands of people before approval, but are then used by millions. **Rare or delayed side effects only show up after launch.** Regulators therefore collect spontaneous reports (FDA: FAERS, 20+ million reports) and look for *signals*: statistical hints that need a human to investigate.
- Enterprise tools for this (Oracle Argus, ArisGlobal) are expensive. Free alternatives are either raw data dumps or one-off notebooks.
- Drug–drug interactions (DDIs) are a second problem: there are far too many drug pairs to test, so new or rarely co-prescribed pairs are **uncharted**. A structure-based model can rank them for expert review.

Who it is built for

Audience	What they do with it	Why this tool fits
Pharmacy / medical students (primary: pilot at colleges)	Learn how signals are detected; explore real FAERS data; check interactions	Free, explains its own methods, labels what is measured vs modelled vs simulated
Pharmacovigilance trainees and junior safety officers	Practise disproportionality analysis, see 2×2 tables and CIs	Shows the underlying a/b/c/d counts, not just a score
Pharmacoepidemiology researchers	Hypothesis generation; cross-check a signal quickly	Reproducible pipeline, versioned data (`data_version`), published benchmarks
Hospital pharmacists (future, Prescription Review phase)	Upload a prescription, confirm the drug list, get an interaction and safety report	Three evidence tiers: documented / model-predicted / no evidence found

BE HONEST ABOUT

It is **not** a clinical tool and never says “safe”. The vocabulary is deliberate: *interaction likelihood score*, *no evidence found*, *signal* (not “risk”), and “research use only”. Expect the question “can a doctor rely on this?”; the answer is no, and that is by design.

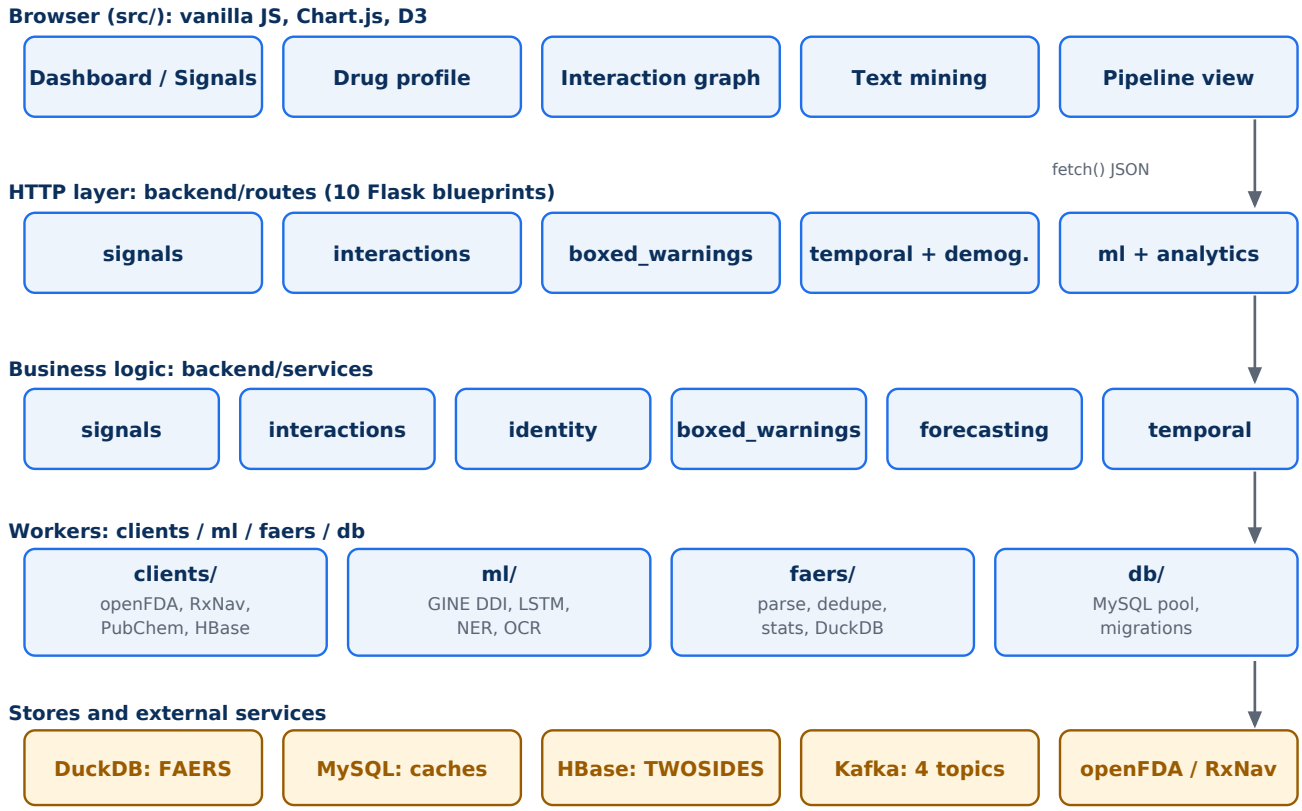
Project facts to memorise

Item	Value
Origin	Big Data mini-project, Manipal Academy of Higher Education (Sreyaan Roy and Sohan Arvind Sanil); since extended well beyond the course brief
Backend	Python 3.11, Flask (10 blueprints, layered routes → services → clients/ml/faers/db), Gunicorn, ~7k lines in backend/
Frontend	Vanilla HTML/CSS/JS (no build step), Chart.js 4.4, D3.js v7, Tippy.js; ~2,500-line app.js

Item	Value
Stores	DuckDB (FAERS analytics), MySQL 8 (app cache, Kafka sinks), HBase (TWOSIDES interactions), Parquet files (staging)
Streaming	Kafka + ZooKeeper, 4 topics, 1 producer service, 1 consumer group
ML	GINE graph network (DDI), 2-layer LSTM (forecast), d4data/biomedical-ner-all (NER), EasyOCR (prescriptions)
DDI data	TDC DrugBank, 1,705 valid drugs, drug-level splits 75/10/15 % with S0/S1/S2 test regimes
Testing	42 pytest tests, including an architecture-layering test and a route-inventory test
Delivery	Docker Compose (Kafka, HBase, app), Caddy reverse proxy, GitHub Actions CI/CD
Status	Milestone 1 (phases 1–4) complete; Milestone 2 (phases 5–11, pilot release) in progress: phase 5 done, phase 6 (real FAERS) code-complete

2. Architecture

PharmaWatch is a **pragmatic modular monolith**: one Flask process serves the API and the static frontend, with strictly layered packages, plus separate worker containers for Kafka. The layering rule is checked by a test (`test_architecture.py`), so it cannot rot silently.



Layer	Responsibility	May import	Why it is separate
routes/	Parse the request, call one service, return JSON. No SQL, no outbound HTTP, no model code	services, core	Thin controllers; easy to test and to keep the route list stable
services/	Business logic (signals, interactions, boxed warnings, forecasting, identity)	clients, ml, faers, db	One place for rules; no Flask objects so unit-testable
clients/	Everything that leaves the process: openFDA, RxNav, PubChem, ClinicalTrials, HBase	core	One HTTP wrapper with retry/cache; swap or mock one dependency
ml/	Model definitions and inference (ddi/, forecast, ner, ocr), lazy-loaded	core	Heavy imports (torch, transformers) stay out of request paths until needed
faers/	Download, parse, de-duplicate, load, and all statistics	core	Single statistics path: every endpoint gets the same 2×2 counts
db/	MySQL connection pool (DBUtils), schema, migrations	core	Connection and schema concerns isolated
ingest/	Kafka producer and consumer (separate processes)	db, faers	Different lifecycle from the web app
validation/	DDI evaluation and calibration, benchmark runners	ml, core	Scientific evidence is code, not a notebook

How a request flows (example: the signal table)

- Browser calls `GET /api/signals` → `routes/signals.py` → `services/signals.get_signals()`.
- The service reads `signals_cache` in MySQL. A cached row is only reused if its `data_version` equals the version of the loaded FAERS quarters (so reloading data invalidates old numbers automatically).
- On a miss it resolves drug → RxNorm ingredient and event → MedDRA Preferred Term (`IdentityService`), runs the 2×2 query in DuckDB (`faers/stats.contingency`), then `compute_metrics`.
- If the local database is unavailable it falls back to live openFDA counts and marks the response `'fallback': True`, because those counts are approximate and not comparable.
- Result is upserted to `signals_cache` and returned; the frontend (`src/api_layer.js`) renders it with Chart.js.

Why a monolith and not microservices?

Team of two, one deployable, request volume of a pilot, and the expensive part (models) wants one warm process anyway. Gunicorn runs **1 worker × 8 threads** so the PyTorch model is loaded once instead of per worker (avoids out-of-memory). The things that genuinely have a different lifecycle (Kafka producer/consumer, HBase) are separate containers. The layering means a service could be lifted out later without rewriting it.

3. Tech stack and the reasoning behind each choice

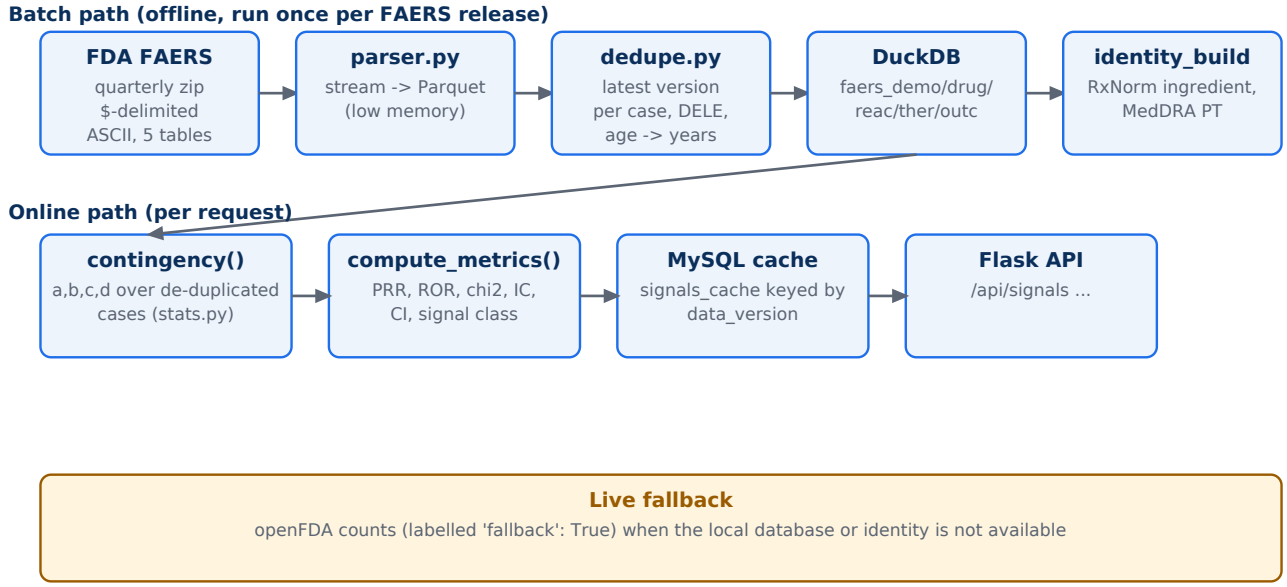
Choice	Why this	What the alternative would have cost / when I would switch
Flask	Small, explicit, blueprints give modularity; ML ecosystem is Python so one language end to end	FastAPI gives async + automatic OpenAPI; worth it if concurrency or typed contracts matter more
DuckDB for FAERS	Embedded columnar OLAP engine: scans millions of rows with vectorised execution, reads Parquet directly, window functions for de-duplication, single file, no server to run	MySQL is row-oriented and would be slow on wide aggregations; Spark is overkill at a few GB on one machine
Parquet staging	Columnar, compressed (snappy), typed; parsing the raw $-delimited text happens once	Re-parsing 1.5 GB of text per query is wasteful; CSV loses types
MySQL 8 (pooled)	Transactional app state: caches, upserts (`ON DUPLICATE KEY UPDATE`), Kafka sinks, concurrent writers	Postgres would be equally fine; SQLite was used in milestone 1 and replaced in phase 3 for pooled, concurrent access by the web app and the Kafka consumer
Kafka	Decouples slow, unreliable external sources from the app; durable log with replay (7-day retention); consumer-group offsets	A cron job would be simpler, but loses replay, back-pressure and the streaming story
HBase (TWOSIDES)	Wide-column store with fast point reads on a row key `DRUGA_DRUGB` and prefix scans for “all partners of drug A”	Honest take: at this size MySQL or DuckDB would also work; HBase is chosen to match the course's Hadoop-ecosystem brief
PyTorch + PyG + RDKit	RDKit parses SMILES to atoms/bonds and fingerprints; PyG implements GINEConv and batched graphs	DGL is equivalent; a pure fingerprint model needs only scikit-learn/XGBoost (and is actually stronger here: see section 7)
Transformers (HuggingFace)	Pre-trained biomedical NER without training data of my own	A CRF or spaCy model is lighter but needs labelled data
Vanilla JS + Chart.js + D3	No build tooling; easy to host as static files; D3 force simulation is ideal for the interaction graph	React/Vue would help with state management; the cost is global-variable coupling and script load order in the current code
Docker Compose + Caddy	One command brings up Kafka, HBase and the app; Caddy gives HTTPS and rate-limiting with a 10-line config	Kubernetes is unnecessary for a single-host pilot
GitHub Actions	Ruff lint, MySQL service container for migrations, pytest, image build, gated deploy	—

4. The big-data side

Be ready to say *which* parts are genuinely distributed and which are not. The five Vs make a good frame:

V	In PharmaWatch
Volume	FAERS: 20+ million cumulative reports; the 20 most recent quarters are ~1.5 GB downloaded and 3–5 GB loaded. DrugBank DDI: ~272k labelled pair rows
Velocity	Four Kafka topics fed by polling openFDA (every ~10 min), PubMed (hourly), ClinicalTrials.gov and FDA MedWatch RSS
Variety	Structured (FAERS tables), semi-structured (JSON APIs, RSS), unstructured (clinical text, Reddit, prescription images), graph (molecules)
Veracity	Spontaneous reports are duplicated, incomplete and biased; hence case de-duplication, identity resolution, and explicit evidence labels
Value	Early safety signals and interaction hypotheses that a human can then investigate

4.1 FAERS ingestion pipeline (batch)

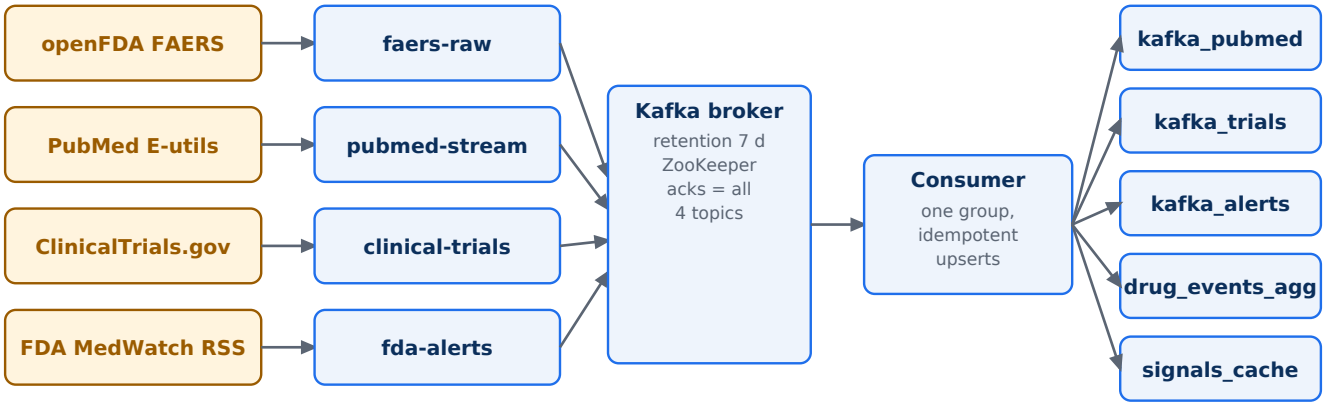


Step	What it does	Why it matters
downloader.py	Fetches quarterly ZIPs from FDA, verifies them, refuses mock files	Provenance: the registry stores a SHA-256 per zip and flags fixtures
parser.py	Streams each of DEMO, DRUG, REAC, THER, OUTC and DELETE members using pyarrow's streaming CSV reader (8 MB blocks, $ delimiter, latin-1) into Parquet	A real DRUG file is ~150 MB of text; streaming keeps memory bounded. Malformed rows are counted, not crashed on
dedupe.py	Keeps one row per **caseid**: latest `fda_dt`, then largest `primaryid` (`ROW_NUMBER() OVER (PARTITION BY caseid ORDER BY ...)`). Drops cases the FDA later deleted. Converts age from YR/DEC/MON/WK/DY/HR to years (0–120 else NULL)	Without this a case with three follow-up versions counts three times and inflates PRR; age in months read as years corrupts demographics
loader.py	Builds a *.building* DuckDB file and atomically swaps it in with `os.replace`; writes the quarter registry	The running app never sees a half-loaded database
identity_build.py	Resolves every drug string to an RxNorm **ingredient** via RxNav (cached, parallel); builds `case_drug` (suspect flag) and a Preferred-Term dictionary	Brand, generic and salt forms of one drug must collapse to one identity or counts are split

KEY IDEA

Single statistics path. Phase 6 made every endpoint call `faers.stats.contingency()` for its 2×2 table. Before that, the dashboard, the consumer and the signal table computed counts in three different ways and disagreed. Say this when asked "what was the hardest engineering problem?": consistency, not speed.

4.2 Kafka streaming pipeline (real-time)



- **Producer** (`ingest/producer.py`): `acks='all'` (leader plus replicas must acknowledge), retries, key = drug name so all messages for one drug go to the same partition (ordering per drug). Polite sleeps keep it under openFDA rate limits.
- **Consumer** (`ingest/consumer.py`): one consumer group, `auto_offset_reset='earliest'`, auto-commit every 5 s, `max_poll_records=50`, a handler per topic.
- **Delivery semantics:** Kafka gives *at-least-once*. Every handler is an **idempotent upsert** (`INSERT ... ON DUPLICATE KEY UPDATE` keyed on pmid / nct_id / alert_key / (drug,event)), so a redelivered message changes nothing. At-least-once + idempotent sink = effectively-once results.
- **Operational telemetry:** each processed message increments `kafka_pipeline_stats`, purged after 7 days; the Pipeline page reads it, so those throughput numbers are real.
- **Why stream at all?** Even though FAERS itself is quarterly, safety alerts, new papers and trial updates are continuous. Kafka lets each source fail and recover independently.

BE HONEST ABOUT

The consumer's `handle_faers` currently computes PRR from the sum of the top-10 event counts and a hard-coded background table. That is not a valid 2×2 table. Milestone 2 (phase 10) removes statistics from the consumer entirely: Kafka only *delivers evidence* and triggers a recompute; all statistics live in `faers/`. If asked, say you found this in your own audit, which is a strength.

4.3 HBase and the TWOSIDES interaction store

- TWOSIDES (Tatonetti et al., 2012) is a public dataset of drug-pair → side-effect associations mined from FAERS. It is loaded by `scripts/load_to_hbase.py` into table `interactions`.
- **Row key design:** `DRUGA_DRUGB` (upper-case). A point lookup tries both orders; a *prefix scan* on `DRUGA_` returns every partner of drug A in one sequential read. HBase sorts rows lexicographically, so the key is the index.
- **Resilience:** the Thrift gateway silently drops idle sockets after ~60 s. `clients/hbase.py` retries once on a stale socket, and a genuine refusal trips a **30-second circuit breaker** so a dead HBase does not stall every request.
- Its role in the product: a pair found in TWOSIDES, or in the FDA label, is tier **documented**.

4.4 What is simulated (say it before they ask)

Spark and HDFS appear in the Pipeline view, but `src/distributed_storage.js` **simulates** their throughput and storage counters. Kafka and HBase are real; Spark Structured Streaming and HDFS are a documented design (`docs/big_data_implementation_guide.md`) that is not deployed. The UI labels the live and simulated parts separately.

> **SAY THIS**
> “At the pilot's scale, a few GB on one node, DuckDB is faster and simpler than Spark. I would introduce Spark when the data outgrows one machine, roughly hundreds of GB, or when I need distributed streaming joins. The Parquet layout and the service boundaries are already shaped so that swap is local.”

5. The statistics: disproportionality analysis

Core idea: if a drug causes an event, that event should make up a *larger share* of the drug's reports than of everyone else's. We build a 2×2 table over de-duplicated cases:

	Event of interest	Other events
Drug of interest	**a**	**b**
All other drugs	**c**	**d**

Measure	Formula	Meaning
PRR (Proportional Reporting Ratio)	(a/(a+b)) / (c/(c+d))	How many times more likely the event is among the drug's reports than among other reports
ROR (Reporting Odds Ratio)	(a·d) / (b·c)	Odds version; has a clean log-normal CI: ln(ROR) ± 1.96·√(1/a+1/b+1/c+1/d)
Chi-square (Yates)	N·(|ad−bc| − N/2)² / ((a+b)(c+d)(a+c)(b+d))	Is the association larger than chance? (≥ 4 ≈ p < 0.05 at 1 d.f.)
IC (BCPNN, WHO-UMC)	log2((a+0.5)/(E+0.5)), E=(a+b)(a+c)/N	Information Component: log of observed/expected; IC025 is the lower credibility bound. IC025 > 0 is a signal

Worked example (from the code's formulas)

a=30, b=970, c=200, d=98,800, N=100,000. Drug share = 30/1000 = 3.0 %. Background share = 200/99,000 = 0.20 %. **PRR = 14.85**, **ROR = 15.28**, **χ^2 = 326**, expected a = 2.3, **IC = 3.45**. Every one of these says “reported far more often than expected”. It still does *not* say the drug causes it.

Signal criteria (Evans et al., 2001, the standard PRR rule)

A pair is a signal when **a ≥ 3 and PRR ≥ 2 and χ^2 ≥ 4**. PharmaWatch grades signals `critical` when additionally ROR lower CI > 1, IC025 > 0 and PRR > 4, otherwise `high`.

Why three metrics?

They fail differently. PRR is intuitive but unstable for small counts. ROR has a well-behaved interval. IC is Bayesian-flavoured and shrinks small-count estimates towards zero, which is why WHO uses it for screening. Agreement across all three is stronger evidence than any one.

Zero cells and continuity

A zero in any cell makes PRR/ROR infinite or undefined, so 0.5 is added to empty cells (Haldane–Anscombe). The project's design note `formulation/DISPROPORTIONALITY_ZERO_CELL_HALDANE_CORRECTION.md` covers this.

BE HONEST ABOUT

Two known defects in `compute_metrics` (listed in the Milestone-2 audit, fixed in phase 7): (1) pairs that fail the signal rule are labelled `moderate` rather than `none`, so a dashboard counting "signals" can over-count; (2) when c = 0 it substitutes an empirical expected value instead of the standard 0.5 correction, which is non-standard. Also, the earlier time-to-onset and demographic "PRR" were derived from invented shares; phase 7 replaces them with real therapy-start dates and per-stratum 2×2 tables. Interviewers love candidates who can name their own bugs and the fix plan.

What disproportionality can not tell you

- **No incidence.** We know reports, not how many people took the drug (no denominator of exposure).
- **Reporting bias:** stimulated reporting after publicity (notoriety bias), the Weber effect (reports peak ~2 years after launch), under-reporting of well-known effects.
- **Confounding by indication:** a diabetes drug will be reported with diabetic complications because patients are diabetic.
- **Masking:** a very common co-reported drug can hide another drug's signal in the background.
- **Report ≠ prescription ≠ causation.** A rising count is not a rising risk. Phase 7 removes wording that implied otherwise.

6. Identity: RxNorm and MedDRA

A statistic is only as good as the entity it counts. FAERS drug names are free text ("COUMADIN", "warfarin sodium", "Jantoven 5 mg"). Events are coded in MedDRA at several levels. PharmaWatch gives every drug and event **one identity**:

- **Drug → RxNorm ingredient.** `IdentityService.resolve_drug` checks the loaded identity table first, then RxNav (NLM's API, cached on disk as JSONL). Combination products like *lamivudine\zidovudine* are split on the backslash and require *all* ingredients to be suspect in the same case.
- **Event → MedDRA Preferred Term (PT)** present in the loaded data: exact match, else a reviewed synonym CSV (`event_synonyms.csv`), else candidate suggestions for a person to confirm. Nothing is guessed.
- **Exposure definition:** a drug counts as exposed only when it is primary or secondary **suspect** in the case, not merely concomitant.
- **Honest empties:** if a drug resolves but has no indexed reports, the API says "not indexed", never a zero count.

SAY THIS

"Before this, the same drug appeared under different names and the watchlist contained terms that are not real MedDRA PTs, like 'Bleeding' or 'QT Prolongation', so counts were silently zero. A test now fails if a watchlist event is not a PT in the data."

7. Machine learning: what, why, and how well

Four models, four different jobs. For each: **what problem, why this model family, how it was trained and judged, and where it is weak.**

Model	Task	Why this family	Where it lives
GINE + Morgan fingerprint	Predict whether two drugs interact, from structure alone	Molecules are graphs; works for brand-new drugs (inductive)	ml/ddi/model.py, scripts/train_gnn_model.py
XGBoost, LogReg, NN-Tanimoto	Same task: baselines	Honest yardsticks; a fancy model must beat them	ml/ddi/baselines.py
2-layer LSTM	Forecast monthly report counts per drug	Sequence model for temporal trends	ml/forecast.py
d4data/biomedical-ner-all	Extract drugs, diseases, symptoms from text	Pre-trained transformer; no labelled data needed	ml/ner.py
EasyOCR (cloud vision planned)	Read prescriptions	Offline fallback for printed text	ml/ocr.py

7.1 Drug–drug interaction model (GINE)

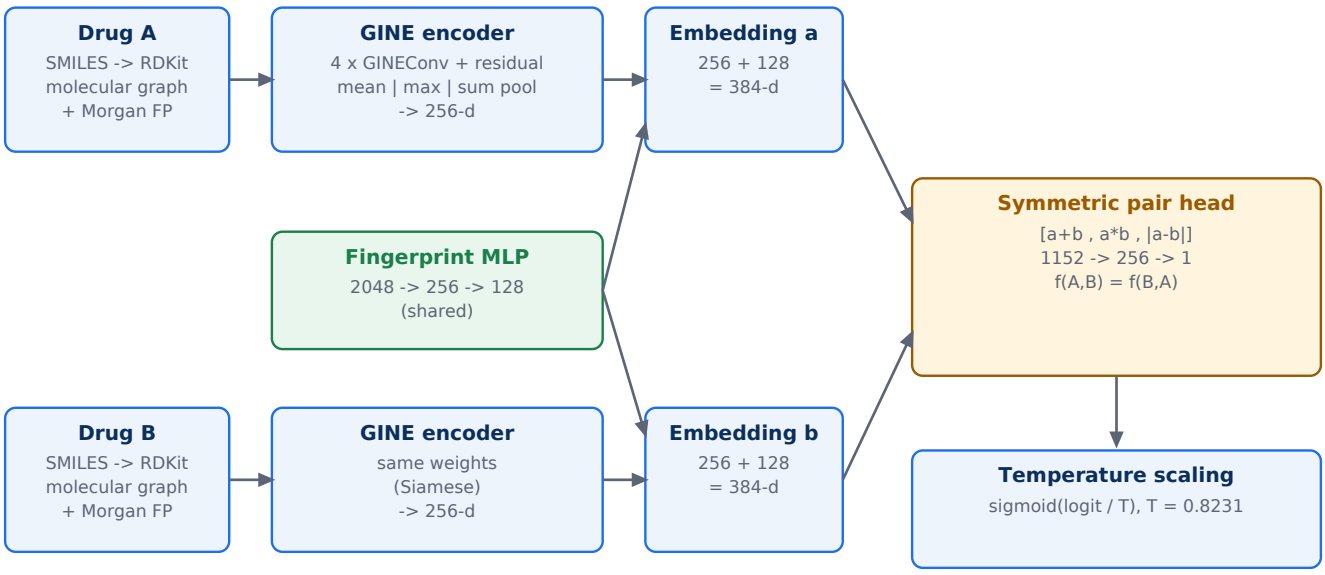
The task in plain words

DrugBank lists ~1,700 approved drugs and which pairs are known to interact. For a **new** drug there is no interaction history, only its chemical structure. So we ask: *from the two molecules' structures, how likely is it that they interact?* This is the "cold-start" problem, and it is why a purely ID-based recommender would not work.

From molecule to tensor

- **SMILES** (text like `CC(=O)Oc1ccccc1C(=O)O`) → RDKit → atoms and bonds.
- **Node features (22-d):** one-hot element (C,N,O,F,S,Cl,P,Br,I,other), degree, formal charge, number of H, aromatic, in-ring, chirality, one-hot hybridisation.
- **Edge features (10-d):** bond type (single/double/triple/aromatic), conjugated, in-ring, stereo (Z/E/none/other).
- **Morgan fingerprint:** radius 2, 2048 bits (ECFP4): a hashed bag of circular substructures around each atom. Fast, strong, and permutation-invariant.

Architecture



Design decision	Reason (the answer to "why?")
GINE instead of plain GCN/GIN	GIN is as expressive as the 1-WL graph test; the **E** adds bond features to each message: $h_v \leftarrow \text{MLP}((1+\varepsilon)\cdot h_v + \Sigma_u \text{ReLU}(h_u + e_{uv}))$. A double bond and a single bond are chemically different, and plain GIN ignores that.
4 layers, hidden 128, residual + BatchNorm	Four hops cover most functional groups (a 4-hop neighbourhood ≈ radius-4 substructure); residuals prevent over-smoothing and ease training
Mean + max + sum pooling	Mean captures composition, max captures the presence of a key motif, sum captures size. Together they make a richer graph vector than any one
Fingerprint branch (2048→256→128)	Fingerprints carry hand-engineered chemistry knowledge; the ablation shows removing it drops S1 AUROC from 0.66 to 0.53
Siamese (shared weights)	Same function embeds both drugs, so it generalises to unseen drugs and has half the parameters
Symmetric head [a+b, a·b, |a−b|]	Interaction is unordered: f(A,B) must equal f(B,A). Sum, product and absolute difference are all permutation-invariant. A plain concatenation [a,b] is not, and the ablation collapses to chance (0.515)
Dedup of molecules per batch	Each unique molecule is encoded once and indexed into pairs; the training loop encodes all train drugs once per step

Data and the three evaluation regimes

Splitting *pairs* at random would leak: the same drug appears in train and test, and the model could just memorise drug identity. So PharmaWatch splits **drugs** (75 % train / 10 % val / 15 % test, seed 42), with hard assertions that no train drug appears in val/test. Then:

Regime	Meaning	Difficulty	Real-world analogue
S0 (warm)	Both drugs seen in training; the *pair* is new	Easiest	Known drugs, new combination
S1 (one unseen)	Exactly one drug never seen	Hard; the primary inductive setting	A new drug against the existing pharmacopoeia
S2 (both unseen)	Neither drug seen	Hardest	Two new drugs

Training recipe

- **Loss:** BCE-with-logits on hard 0/1 labels. **Optimiser:** AdamW (lr 1e-3, weight decay 1e-5) with cosine annealing to 1e-6.
- **Negatives:** DrugBank only lists *positives*. An unlisted pair is **unlabelled**, not "safe" (this is the positive-unlabelled learning problem). Each epoch draws fresh 1:1 negatives from unlabelled train-drug pairs, so the model cannot memorise a fixed negative set.
- **Selection:** best checkpoint by **S1 validation AUPRC** (the cold-start metric), early stopping patience 5, five seeds (42, 123, 456, 789, 1011) reporting mean ± std.
- **Calibration:** post-hoc **temperature scaling**: divide logits by one learned T, fitted by minimising NLL. T = 0.8231. It cannot change ranking, only confidence.

Metrics: say what each one answers

Metric	Question it answers	Caveat here
AUROC	If I pick a random positive and a random negative pair, how often is the positive scored higher?	0.5 is chance. Insensitive to class balance
AUPRC	How good is precision across recall levels?	Test sets are ~1:1 by construction, so it is not real-world precision (true prevalence differs)
Recall@k	Of all true interactions, what fraction is in my top-k?	Saturated: S1 has ~47k positives, so k=100 can only reach ~0.2 %. Phase 8 replaces it with Precision@k

Metric	Question it answers	Caveat here
ECE	Do predicted probabilities match observed frequencies?	Lower is better; reliability diagram in reports/

Results (5-seed validation: AUPRC 0.616 ± 0.014; test below)

Regime	GINE AUROC	LogReg	XGBoost	NN-Tanimoto
S0 warm	0.689	**0.868**	0.825	0.793
S1 one unseen	0.662	0.694	**0.757**	0.751
S2 both unseen	0.637	0.585	**0.671**	0.577

Ablation (S1)	AUROC	Reading
Full GINE	0.662	reference
GIN (no edge features)	0.672	No evidence edge features helped
GINE without fingerprint	0.526	Fingerprint carries almost all the signal
Concatenation head	0.515	Order-dependent head fails

BE HONEST ABOUT

The deployed GINE is beaten by XGBoost (and sometimes logistic regression) in all three regimes. Do not hide this. A strong answer: "I benchmarked honestly, found my deep model did not win, and diagnosed why." The likely reasons, which are documented in the audit:

• **Undertraining:** the loop does one full-batch optimiser step per epoch, ~15 updates in total; the unused `--batch_size` flag is a bug. Best epochs were 3–5, i.e. the model had barely started.

• **Small data:** 1,705 molecules is tiny for a GNN, while fingerprints already encode the relevant substructures.

• **Signal is substructure-level:** many DrugBank interactions follow from class membership (for example CYP inhibitors, QT-prolongers), which fingerprints capture directly.

• **Weak ablations:** the three ablations were not separately *trained* models (phase 8 retrains them properly).

• **Calibration did not help:** ECE went 0.0768 → 0.0780 and was fitted and scored on the same early-stopping set; the API still hard-codes `calibrated: True`. Phase 8 fixes the protocol and the flag. Mitigation already in place: the API presents the output as an *interaction likelihood score* in a three-tier scheme, and the roadmap's fallback is to deploy XGBoost as the modelled tier.

Three-tier output (what the API actually returns)

Tier	Rule	Wording
documented	Found in the openFDA label text or in TWOSIDES (HBase)	"Documented clinical interaction found in regulatory labels or databases"
model_predicted	Not documented, score ≥ 0.50	"Hypothesis-generating prediction based on molecular graph similarity"
no_evidence_found	Not documented, score < 0.50	"No evidence found", never "safe"

Why tiers? Because the model should never *override* curated knowledge, and a missing record is absence of evidence, not evidence of absence.

7.2 LSTM report-trend forecaster

- **Input:** monthly adverse-event report counts for one drug from openFDA (YYYYMM buckets), min-max normalised. **Window:** 30 steps → predict the next.
- **Model:** 2-layer LSTM (hidden 64, dropout 0.2) + linear head; MSE loss, Adam lr 1e-3, 100 epochs, chronological 80/20 split (never shuffled; shuffling would leak the future). One model per drug (metformin, metoprolol succinate, montelukast, pantoprazole, warfarin are trained).

- **Use:** plot actual vs predicted “baseline”; flag a rising slope; mark the boxed-warning date; the “warning gap” scan classifies HIGH/MODERATE/LOW/MINIMAL.
- **Why an LSTM?** Gated recurrence handles variable-length sequential dependence without hand-built lag features; it was a natural fit for a course on sequence data. For one short univariate series a classical model (ARIMA / ETS / Prophet) is a legitimate and often better baseline.

BE HONEST ABOUT

Weaknesses you should volunteer: (1) it forecasts *raw counts*, so overall FAERS growth looks like a rising drug risk; the fix is to model the drug's *share* of all reports; (2) the plotted baseline is computed in-sample, so it looks better than it is; (3) `rising_signal = slope > 0.5 % of max` is a heuristic, not a statistical test; (4) no out-of-sample benchmark against a naive or moving-average forecast. Phase 7 reframes it as descriptive and adds an out-of-sample baseline.

Z-score spike detector (the simple model that actually works)

`zscore_spikes`: for each month, compare to the mean and standard deviation of the trailing 12 months; flag $z > 2$. Needs ≥ 3 months of history; zero variance is treated as std = 1. It is cheap, explainable and has unit tests. Good example of “use the simplest thing that is defensible”.

7.3 Biomedical NER (text mining)

- **Task:** token classification: label spans in free text (clinical notes, Reddit posts) as `Medication`, `Disease_disorder`, `Sign_symptom`...
- **Model:** `d4data/biomedical-ner-all` via HuggingFace `pipeline('ner', aggregation_strategy='simple')`; ~260 MB; lazy-loaded on first request. *aggregation_strategy* merges word-piece tokens back into whole entities.
- **Post-processing:** strip punctuation, drop tokens under 3 characters or in a clinical stop-word list.
- **How it feeds the system:** every (drug, event) pair found in a text is stored in `drug_events` with confidence = $\min(\text{score}_{\text{drug}}, \text{score}_{\text{event}})$.

BE HONEST ABOUT

The UI calls it “BioBERT”; it is not. The checkpoint is DistilBERT-based. Say “a DistilBERT-based biomedical NER model; the UI label is being corrected”. Also: pairing is the *cross product* of all drugs and events in a text, with **no negation handling** (“no nausea with aspirin” still links aspirin–nausea), and the NER benchmark on an annotated ADE corpus is planned (phase 8), not done. Social-media mining is labelled exploratory and never feeds the FAERS statistics.

OCR and prescription review (roadmap)

EasyOCR (CRNN-style detector + recogniser) is wired in as the offline fallback. Milestone 2's phase 9 adds a cloud vision extractor behind a swappable interface, a **mandatory user-confirmation step** before any analysis, and a no-storage-by-default privacy rule. Only synthetic prescriptions are used until ethics approval.

8. Engineering quality: how it is kept honest

Practice	What it is	Why it matters
Layering test	`test_architecture.py` parses imports with `ast` and fails if a route imports db/requests/torch/duckdb/clients/ml/faers, a service imports Flask, a lower layer imports upward, or anything edits `sys.path`	Architecture rules enforced by CI, not by good intentions
Route inventory snapshot	`route_inventory.json` lists every endpoint with status and top-level keys	Lets a big refactor (phase 5: move/delete only) prove no behaviour changed
Refactor diff harness	`scripts/refactor_check` compares endpoint bodies and SQL before/after	Behaviour-preserving restructuring with evidence
Fixture-based FAERS tests	Tiny synthetic quarter proves de-duplication, deleted cases, age-unit conversion	Statistical code gets golden tests; the roadmap adds hand-worked 2×2 cases
Zero-leakage assertions	Train/val/test drug sets asserted disjoint at split time	Prevents the classic inflated-AUC mistake
Single source of truth	One admin-key decorator, one cache helper, one z-score, one HBase lookup, one 2×2 path	Duplicates had diverged and disagreed
Evidence labelling	Every result will carry `evidence` = measured | modelled | simulated, plus `data_version` and `model_version`	A reader can always tell a number from a guess
Atomic data swap	Build DuckDB beside the live file, then `os.replace`	No half-loaded database ever served
Resilience	HBase circuit breaker, bounded retries on HTTP, lazy model loading, graceful rate-limiter fallback	A dead dependency degrades one feature, not the app

Security and platform: known gaps (and the plan)

- No user accounts or roles yet; the admin key is accepted in the query string (should be header-only). Phase 11 adds roles, an append-only audit log, upload size/rate limits.
- `torch.load` for the GNN checkpoint lacks `weights_only=True` (the LSTM loader has it): pickle can execute code. Easy fix; mention you know.
- Some endpoints return raw exception text; to be replaced with generic errors + server-side logging.
- CORS is restricted to localhost origins by `ALLOWED_ORIGINS`; a rate limiter (10,000/h default) sits in front; Caddy adds TLS in production.

What is real vs. what is pending right now

Area	Status
FAERS parse → de-dup → DuckDB → identity → single stats path	Built and tested. **Only one real quarter (2026Q2) is loaded**; the 20-quarter load is waiting on the FDA host (connection resets)
Signal statistics correctness (golden tests, “none” class, real time-to-onset)	Planned, phase 7
Validation harness (OMOP / EU-ADR reference sets, NER benchmark, DDI retrain)	Planned, phase 8
Prescription review, drug profile page, Kafka as evidence-only, accounts/audit	Planned, phases 9–11
Kafka + HBase + MySQL + Docker stack, GNN, LSTM, NER, dashboard	Working (with the caveats above)

9. Your 10 weak spots: own them before they find them

Interviewers probe the thinnest part of a project. Each line is: **flaw → why it happened → what you did or would do.**

#	Weak spot	How to talk about it
1	GINE loses to XGBoost in S0/S1/S2	Honest benchmark was the goal; diagnosed undertraining (15 updates) and tiny data; roadmap retrains with mini-batches and can deploy XGBoost as the modelled tier
2	Calibration did not reduce ECE; `calibrated: True` hard-coded	Fitted and scored on the same set; fix is a separate calibration fold and a flag that is true only when ECE improves
3	Ablations were not trained models	Time constraint; phase 8 trains each variant separately
4	Spark/HDFS are simulated	Kafka/HBase are real; DuckDB is right-sized; design for Spark exists and is labelled
5	compute_metrics labels non-signals “moderate”; c=0 handling non-standard	Found by audit; phase 7 adds a none class and golden tests against hand-worked tables
6	Kafka consumer computes PRR from top-10 counts	Invalid denominator; phase 10 makes Kafka evidence-only and moves stats into faers/
7	NER mislabelled as BioBERT; no negation; cross-product pairing	Correct the label; add negation (NegEx-style) and an annotated-corpus benchmark
8	LSTM on raw counts, in-sample baseline	Normalise by total reports, out-of-sample baseline, descriptive wording
9	Only 1 real FAERS quarter loaded so far	Pipeline is built and tested on fixtures and one real quarter; download is gated by the FDA host
10	No auth, raw errors, `torch.load` without weights_only	Known and scheduled in phase 11

10. Interview question bank

Answers are written in the first person so you can adapt them. Read the **Trap** lines twice: they are where candidates lose points. Questions are grouped from easy (project and motivation) to hard (ML and statistics internals).

A. Project, motivation, ownership

Q1. Walk me through PharmaWatch. Who is it for and why does it exist?

It is a pharmacovigilance dashboard. Regulators collect spontaneous adverse-event reports, but exploring them takes expensive software or custom analysis. PharmaWatch loads FAERS, de-duplicates it, applies standard disproportionality statistics, adds a structure-based interaction model and text mining, and presents it so a student or junior safety officer can see *both* the result and how it was obtained. Primary audience is pharmacy and medical colleges (a pilot), secondary is researchers.

Likely follow-up: Why not just use the FDA's own FAERS dashboard?

Trap: Do not claim it replaces regulatory tooling or is clinically validated.

Q2. What was your personal contribution?

Answer concretely: name the parts you built or can explain end to end (for example the FAERS pipeline and de-duplication, the statistics path, the GINE model and benchmark, or the Kafka ingest), and say who on the team did what. Then add one decision you made and defended.

Trap: Vague ownership. Pick two components and know them to the line level (section 4 and 7 of this guide).

Q3. What was the hardest problem you solved?

Consistency of the numbers. The dashboard, the Kafka consumer and the signal table each computed counts differently and disagreed. I introduced a single `contingency()` function over de-duplicated cases and made every endpoint use it, then added a test that all endpoints return identical a/b/c/d for the same pair.

Q4. What would you do differently if you started again?

Start with the validation harness and real data first. I built UI and models before I had trustworthy denominators, and later had to re-audit invented numbers (time-to-onset, demographic PRR, background counts). Evidence-first would have saved a phase.

Q5. How do you know it works?

Layers of evidence: golden/fixture tests for the statistics and de-duplication, leakage assertions for the ML split, a benchmarked DDI model on three cold-start regimes, and (planned) agreement with published reference sets (OMOP, EU-ADR). I would say “benchmarked against published reference standards”, never “medically validated”.

B. Architecture and design

Q6. Describe the architecture.

A layered monolith: static frontend, Flask blueprints that only parse and format, services with business logic, and below them clients (external APIs), ml, faers (ingest and statistics) and db. Kafka producer/consumer, HBase and MySQL run as separate containers. An `ast`-based test enforces the layering.

Q7. Why a monolith?

Two developers, one deployable, pilot-scale traffic, and heavy models that you want loaded once. Microservices would add network, deployment and observability cost with no benefit yet. The layering means I can extract a service later.

Likely follow-up: At what point would you split it?

Trap: “Microservices are better” without a reason. Tie the answer to scale, team size and failure isolation.

Q8. Why DuckDB for FAERS rather than MySQL or Spark?

FAERS analysis is OLAP: scans and aggregations across millions of rows with joins and window functions. DuckDB is columnar and vectorised, reads Parquet natively, runs in-process and ships as one file. MySQL is row-oriented and tuned for transactions; Spark adds cluster overhead for data that fits on one machine.

Q9. Why is there both MySQL and DuckDB? Isn't that two databases for one job?

They do different jobs. DuckDB holds the immutable analytical dataset (rebuilt per FAERS release, opened read-only). MySQL holds mutable application state: signal caches, Kafka sinks, predictions, with concurrent writers and upserts. Separating read-heavy analytics from transactional state is a standard pattern.

Q10. How does caching work, and how do you avoid serving stale numbers?

Computed signals are stored in `signals_cache` with a `data_version` string built from the loaded quarters (for example “FAERS 2021Q3-2026Q2 (20 quarters, N cases)”). A cached row is only reused when its version matches the current one, so loading new data invalidates it automatically. Pair predictions are cached under a canonically sorted drug-pair key so (A,B) and (B,A) hit one row.

Q11. What happens when an external dependency is down?

Each failure is contained. HBase has a 30 s circuit breaker plus one retry for stale sockets. HTTP calls go through one wrapper with retry and timeouts. Models are lazy-loaded and the service reports unavailability. openFDA is a labelled fallback for the local database, with `fallback: True` so nobody mistakes approximate counts for the real ones.

Q12. How is the system tested?

42 pytest tests: statistics and de-duplication on a synthetic FAERS fixture, identity resolution, architecture rules, a route-inventory snapshot, DDI dataset leakage and model symmetry, calibration, z-score spikes, MySQL routes, and the “all endpoints report identical counts” test. CI runs ruff, applies migrations to a MySQL service container and runs the suite.

Likely follow-up: What isn't tested?

Trap: Saying “everything”. Admit: compute_metrics golden tests and the validation benchmarks are phase 7–8.

Q13. How would this scale 100×?

Data: move FAERS staging to partitioned Parquet in object storage and query with DuckDB/ClickHouse, or Spark if it truly exceeds one node. Stream: more Kafka partitions (key by drug) and more consumers in the group. Serving: multiple stateless web workers behind Caddy/nginx with models in a separate inference service and a shared cache (Redis). MySQL read replicas for caches.

Likely follow-up: What breaks first?

Trap: Gunicorn 1 worker × 8 threads is a deliberate memory choice; say how you would separate inference to scale it.

C. Big data, Kafka, HBase

Q14. Explain Kafka in this project. Why is it there?

It decouples four unreliable external sources (openFDA, PubMed, ClinicalTrials.gov, MedWatch RSS) from the application. Producers publish to topics; a consumer group reads, and writes idempotently to MySQL. Benefits: buffering, replay for 7 days, independent failure and recovery, and consumer lag as an operational metric.

Q15. What delivery guarantee do you have? How do you handle duplicates?

At-least-once. Consumers use auto-commit, so a crash can redeliver. Every handler is an idempotent upsert keyed on natural keys (pmid, nct_id, alert_key, drug+event), so duplicates are harmless.

Likely follow-up: Could you lose a message?

Trap: Auto-commit may commit an offset before processing finishes; a crash in between can skip it. The fix is manual commit after the write. Acknowledge this.

Q16. Why key messages by drug?

Kafka orders messages only within a partition. Keying by drug sends all of one drug's messages to the same partition so per-drug ordering is preserved while different drugs process in parallel.

Q17. What does acks='all' mean?

The producer waits until the partition leader and all in-sync replicas have written the record, which gives the strongest durability at some latency cost. In the single-broker dev setup replication factor is 1, so it matters more in production.

Q18. Why HBase for the interaction table? Isn't that overkill?

Access pattern: look up a pair by key, or list all partners of one drug. The row key `DRUGA_DRUGB` makes both a single read or a prefix scan, and HBase scales that horizontally. Honestly, at this size MySQL would work too; HBase was chosen to implement the Hadoop-ecosystem part of the brief and I treat it as a read-only reference store.

Q19. Where are Spark and HDFS?

They are a documented design and a simulated view, clearly labelled. The data fits on one node, so DuckDB over Parquet is faster and simpler. Spark would come in at hundreds of GB or for distributed streaming aggregations.

Trap: Never present simulated metrics as real. Say it first.

Q20. Why Parquet?

Columnar and compressed, typed, splittable, readable by DuckDB/Spark/pandas. A column scan reads only the columns needed, which is what aggregations over FAERS do.

Q21. How do you load 20 quarters without running out of memory?

Streaming parser: pyarrow reads each zip member in 8 MB blocks and appends to Parquet via a writer, so memory is bounded by the block, not the file. Quarters are loaded sequentially into a database built beside the live file, then swapped atomically.

Q22. What is the FAERS de-duplication rule and why does it matter?

A case can be re-submitted as follow-ups with the same CASEID and new PRIMARYIDs. Keep the version with the latest FDA date (tie-break largest PRIMARYID), and drop cases listed in the DELETE files. Without it the same patient is counted several times, inflating a and distorting PRR.

D. Statistics and pharmacovigilance

Q23. Define PRR and explain it to a non-statistician.

Among all reports that mention this drug, what share involve this event, compared with the share among all other reports. A PRR of 3 means the event is reported three times as often with the drug as without it.

Q24. When is something a “signal” in your system?

$a \geq 3$, PRR ≥ 2 and $\chi^2 \geq 4$ (Evans criteria). Then “critical” if also ROR lower CI > 1, IC025 > 0 and PRR > 4; otherwise “high”.

Q25. Why compute PRR, ROR and IC? Aren't they redundant?

They correlate but fail differently. PRR is intuitive but unstable at small counts; ROR has a standard confidence interval; IC (BCPNN) shrinks sparse cells and is the WHO's screening measure. Agreement is stronger evidence.

Q26. A drug has PRR = 12. Does it cause the event?

No. Disproportionality is a hypothesis generator. It cannot see exposure denominators, is confounded by indication and co-medication, and is affected by notoriety and Weber-effect reporting. A signal triggers clinical review, case series and pharmacoepidemiology.

Trap: Any causal wording. Say “reported disproportionately often”.

Q27. What is confounding by indication? Give an example.

The disease being treated causes the event, not the drug. Insulin is reported with hypoglycaemia and also with diabetic complications; chemotherapy with death. The drug and event co-occur because of the patient's condition.

Q28. What is the Weber effect and notoriety bias? Where do they appear in your project?

Weber effect: reporting peaks in the 1–3 years after launch and then declines regardless of true risk. Notoriety bias: publicity or a regulatory action increases reports. The boxed-warning analysis has a three-tier Weber detector (exact window, historical proxy, relative pattern) to caution against reading post-warning report surges as ineffective warnings.

Q29. How do you handle zero cells?

Add 0.5 to empty cells (Haldane–Anscombe) so ratios and logs are defined. I also know the current code has a non-standard substitution when c = 0, listed in my audit with a planned fix and golden tests.

Q30. Why a chi-square with Yates correction?

Chi-square tests association in a 2×2 table; Yates subtracts N/2 to correct for the discreteness of counts and is conservative for small expected values. Threshold ≥ 4 corresponds to $p < 0.05$ for one degree of freedom.

Q31. What are the denominators problems in FAERS?

Reports are not exposures. The “all other reports” group is not a population at risk and is affected by what else gets reported. That is why I call them reporting ratios and why I removed invented background tables.

Q32. Why drug identity resolution at all?

“Coumadin”, “warfarin sodium” and a typo are one drug. If you do not collapse them, a is split across names and PRR is biased low. RxNorm ingredient is the standard key; MedDRA PT is the standard event key.

E. Machine learning: DDI model

Q33. Why a graph neural network for drugs?

A molecule is naturally a graph: atoms are nodes, bonds are edges. A GNN learns substructure features by passing messages along bonds, and a shared encoder gives representations for drugs never seen at training, which is the cold-start requirement.

Q34. What does GINE add over GIN?

Edge features. Each message is ReLU($h_u + e_{uv}$) before summation, so bond type, aromaticity and stereo influence the update. GIN treats every bond identically.

Q35. Why is the pair head symmetric? What happens if it is not?

Interaction is unordered. [a+b, a·b, |a−b|] is permutation-invariant, so f(A,B) = f(B,A) exactly. With plain concatenation the model learns an order-dependent function; in my ablation the concatenation head fell to AUROC 0.515, essentially chance.

Likely follow-up: How is symmetry tested?

Q36. Why drug-level splits instead of random pair splits?

Random pair splits leak: the same drugs appear in train and test, and the model can memorise per-drug tendencies, giving inflated scores that do not hold for new drugs. Drug-level splits measure what we actually care about, performance on unseen molecules.

Q37. What are S0, S1 and S2?

S0: both drugs seen, the pair is new. S1: exactly one drug unseen. S2: both unseen. Difficulty increases S0 → S2. S1 is the primary metric because a new drug paired with the existing pharmacopoeia is the real use case.

Q38. How did you create negatives?

DrugBank only gives positives. I sample unlabelled pairs among train drugs, fresh 1:1 each epoch, excluding known positives. These are *unlabelled*, not confirmed non-interactions, so some are noisy.

Likely follow-up: What are the consequences?

Trap: Calling them “non-interacting pairs” or “safe”. It is positive-unlabelled learning.

Q39. Your GNN lost to XGBoost. Why, and why keep it?

Likely reasons: undertrained (one full-batch step per epoch, ~15 updates, best epochs 3–5), very small dataset (1.7k molecules), and a task where fingerprints already encode the discriminative substructures. The fingerprint-free ablation scoring 0.526 also suggests the graph branch alone was barely trained.

Why keep it: it is the research component; it is labelled as an experimental score in a three-tier output; and the benchmark itself is a deliverable. The roadmap includes mini-batch retraining and deploying whichever model wins S1 AUPRC (XGBoost as fallback).

Trap: Making excuses or claiming the GNN is better. State the result plainly, then the diagnosis.

Q40. What is temperature scaling and why did ECE not improve?

Divide logits by a learned scalar T so confidences are rescaled without changing ranking. T was 0.82 (model slightly underconfident). ECE moved 0.0768 → 0.0780 because I fitted T on the same set I used for early stopping and scored it there too, and a single parameter cannot fix miscalibration that is not a pure scale error. Fix: separate calibration fold, and report ‘calibrated’ only if ECE improves.

Q41. Explain AUROC vs AUPRC. Which do you trust here?

AUROC is rank quality over all thresholds; AUPRC is precision vs recall. My test sets are 1:1 by construction, so neither reflects real-world prevalence. For ranking candidate interactions for review, precision@k is the operational metric; Recall@k saturated because there are ~47k positives.

Q42. How would you improve the DDI model?

Mini-batch training with proper epochs; pretrain the encoder (masked-atom or contrastive, or use pretrained molecular embeddings); add target/enzyme/pathway side information as extra node or edge types; multi-task prediction of interaction *type* (DrugBank has types) instead of binary; leakage-aware hard-negative mining; and ensemble with the fingerprint model.

Q43. Could the model hurt a patient?

That is why it is research-only and tiered. It never outputs “safe”; a low score yields “no evidence found”. Documented sources override the model. Any clinical use would need prospective validation and regulatory process.

F. Machine learning: LSTM, NER, OCR

Q44. Why an LSTM for forecasting? Is that the best choice?

It was a reasonable sequence model for the course, but for one short monthly series a seasonal ARIMA/ETS or Prophet baseline can match it with less overfitting risk. The right engineering step is an out-of-sample comparison against a naive and a moving-average baseline, which is on the roadmap.

Q45. How did you avoid leakage in time-series evaluation?

Chronological 80/20 split with no shuffling and normalisation constants taken from the training window stored in metadata. Honest caveat: the displayed baseline was computed in-sample.

Q46. What is the vanishing-gradient problem and how does an LSTM address it?

In plain RNNs gradients shrink exponentially through time steps. An LSTM has a cell state with additive updates controlled by forget/input/output gates, giving a nearly linear gradient path (the constant error carousel).

Q47. How does the NER model work?

A transformer encoder produces a contextual embedding per sub-word token; a linear layer classifies each token into BIO entity tags (such as Medication, Sign_symptom). The pipeline's ‘simple’ aggregation merges consecutive tokens of one entity into a span with an averaged score.

Q48. Is it BioBERT?

No. The checkpoint is d4data/biomedical-ner-all, a DistilBERT-based model fine-tuned on biomedical text. The UI label said BioBERT and is being corrected. DistilBERT is a distilled, smaller BERT (about 40 % fewer parameters, faster).

Trap: Bluffing. This is a known audit finding.

Q49. How do you turn NER output into drug–event pairs? What's wrong with it?

Currently the cross product of drugs and events in one text, with confidence = min of the two scores. It ignores negation and distance, so it over-links. Better: relation extraction or rule-based negation (NegEx) and sentence-level windowing, evaluated on an ADE corpus.

Q50. Why not send prescriptions to a cloud LLM straight away?

Privacy and control. Phase 9 uses a cloud vision extractor behind an interface, but only after informing the user, never storing the image by default, stripping identifiers, and requiring a human confirmation step before any analysis. Offline OCR remains the fallback.

G. Engineering judgement and behavioural

Q51. Tell me about a time you found your own work was wrong.

The Milestone-2 audit: I found invented numbers in the time-to-onset and demographic outputs, a hard-coded background table feeding PRR, and a watchlist of non-PT event names. I wrote the evidence into a context document, changed the rule to ‘every number is computed from data or absent’, and scheduled each fix with a test.

Q52. How do you decide between a quick fix and the right fix?

Phase 5 is an example: it was a behaviour-preserving refactor protected by a route inventory and a diff harness, so I could move and delete aggressively without risking regressions. Features that already compute real results are not changed in the same pass.

Q53. How do you make sure the UI does not mislead?

Controlled vocabulary (signal, interaction likelihood score, no evidence found), evidence badges (measured / modelled / simulated), data-version stamps, explicit fallbacks and a research-use disclaimer. The rule is also tested: no endpoint may return a number when its source data is empty.

Q54. What are the ethical issues?

Misuse as clinical advice; false reassurance from “no evidence”; privacy of prescription uploads; biases in spontaneous-reporting data (country, drug popularity). Mitigations: wording, tiers, no-storage default, synthetic data until ethics approval.

Q55. If you had two more weeks, what would you do?

Finish the 20-quarter load; golden tests for compute_metrics and the ‘none’ class; run the OMOP/EU-ADR reference benchmark; retrain the DDI model with mini-batches and real ablations; remove statistics from the Kafka consumer; add authentication and the audit log.

H. Rapid-fire (one-line answers)

Question	Answer
What does FAERS stand for?	FDA Adverse Event Reporting System; spontaneous reports from clinicians, patients, manufacturers
What is MedDRA PT?	Preferred Term: the standard level of the Medical Dictionary for Regulatory Activities used for events
What is RxNorm?	NLM's normalised drug vocabulary; ingredient-level concepts used for identity
What is TWOSIDES?	Public dataset of drug-pair side-effect associations mined from FAERS (Tatonetti 2012)
What is SMILES?	Text notation for a molecule's structure
What is a Morgan / ECFP4 fingerprint?	Hashed circular substructures to radius 2; 2048-bit vector
What is OLAP vs OLTP?	Analytical scans/aggregations vs many small transactional reads/writes
What is an idempotent consumer?	Processing a message twice gives the same final state
What is a circuit breaker?	After failures, stop calling the dependency for a cool-off period and fail fast

Question	Answer
Why 1 Gunicorn worker?	Each worker would load the PyTorch model; threads share one copy
Positive-unlabelled learning?	Only positives are labelled; the rest may contain hidden positives
What is inductive vs transductive?	Inductive generalises to unseen nodes/drugs; transductive only to nodes present at training
Over-smoothing in GNNs?	Deep message passing makes node embeddings converge; residuals and shallow depth mitigate
What does temperature scaling not change?	Ranking and therefore AUROC

11. Cheat sheet: numbers and names

Topic	Facts
Data	FAERS 20+ M reports; 20 quarters target (2021Q3–2026Q2), ~1.5 GB download, 3–5 GB kept; 1 real quarter loaded so far. DrugBank/TDC: 1,705 drugs; train 95k pos pairs, val 52k rows, S0 20k, S1 94.8k, S2 10.5k rows
GINE	22-d nodes, 10-d edges, hidden 128, 4 layers, residual, BN, dropout 0.1; mean+max+sum → 256; FP 2048→256→128; embedding 384; head 1152→256→1
Training	AdamW 1e-3, wd 1e-5, cosine to 1e-6, BCE, 15 epochs, patience 5, seeds 42/123/456/789/1011, val AUPRC 0.6155 ± 0.0138
Test AUROC (GINE / LR / XGB / NN)	S0 0.689 / 0.868 / 0.825 / 0.793; S1 0.662 / 0.694 / 0.757 / 0.751; S2 0.637 / 0.585 / 0.671 / 0.577
Calibration	T = 0.8231; ECE 0.0768 → 0.0780 (no improvement)
LSTM	1 → 64 hidden × 2 layers → 1; window 30; MSE; Adam 1e-3; 100 epochs; 80/20 chronological
Z-score spikes	window 12, min history 3, threshold z > 2
Signal rule	a ≥ 3, PRR ≥ 2, χ^2 ≥ 4 (Evans 2001); critical adds ROR_low > 1, IC025 > 0, PRR > 4
Kafka	Topics: faers-raw, pubmed-stream, clinical-trials, fda-alerts; retention 7 d; group pharmawatch-consumer-group; max_poll 50; auto-commit 5 s; producer acks=all
HBase	Table interactions, row key DRUGA_DRUGB, Thrift on 9090, 30 s circuit breaker
Serving	Gunicorn 1 worker × 8 threads, timeout 120 s; Caddy reverse proxy; rate limit default 10,000/h
Tests	42 pytest tests; ruff E9,F63,F7,F82; MySQL 8 service in CI
Phases	M1: 1 GNN fix, 2 hardening, 3 modularisation + MySQL, 4 DDI rigour. M2: 5 clean architecture ✔, 6 real FAERS (code ✔, load pending), 7 stats correctness, 8 validation, 9 prescription review, 10 coherence + Kafka, 11 platform + pilot

Glossary

Term	Meaning
ADR / ADE	Adverse drug reaction / event
Pharmacovigilance	Detecting, assessing and preventing adverse effects of medicines after approval
Signal	Reported information suggesting a possible new association needing investigation
Disproportionality	Event reported more often with a drug than expected from the rest of the database
Boxed warning	FDA's strongest label warning for serious risks
Cold start	Predicting for items (drugs) with no training history
AUROC / AUPRC	Area under ROC / precision-recall curves
ECE	Expected Calibration Error
GINE	Graph Isomorphism Network with Edge features
NER	Named Entity Recognition
PU learning	Positive-unlabelled learning
Idempotent	Repeating an operation leaves the same result

Last-night checklist

- Say the 30-second pitch aloud twice; then the 2-minute version ending in limitations.
- Be able to draw: system layers, the FAERS pipeline, the GINE diagram (sections 2, 4, 7).
- Compute a PRR by hand from a 2×2 table; explain why it is not causation.

- Know the table of GINE vs baselines and the *reason* it loses; do not be defensive.
- Know exactly what is simulated (Spark, HDFS, pipeline counters) and what is real (Kafka, HBase, FAERS pipeline).
- Have one story each for: a bug you found, a trade-off you made, and a thing you would redo.
- Open the repo once and be ready to point to: `faers/stats.py`, `faers/dedupe.py`, `ml/ddi/model.py`, `ingest/consumer.py`.